

Inteligência Artificial

Assignment 1 - Topic 1E

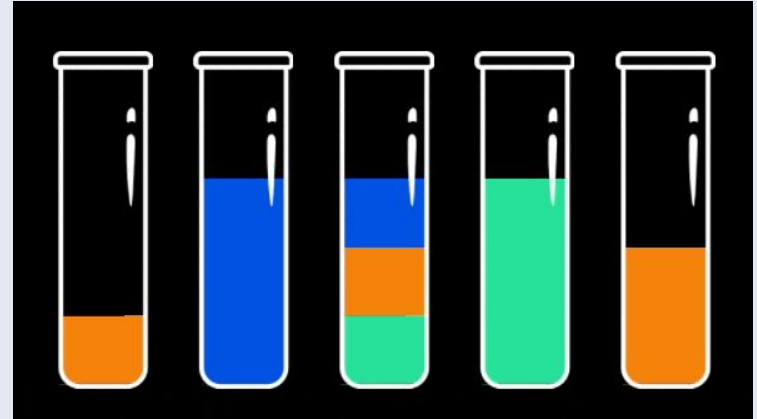
T09_G04:

Francisco Castro Monteiro de Sousa Magalhães
Inês Rocha de Oliveira
Maria Carvalho Lopes

1. Problem Specification

Water Sort Puzzle

A single-player puzzle game where colored liquids are distributed among several tubes. Each tube has a fixed capacity and contains layers of different colors. The objective is to sort the liquids so that each tube contains only a single color.



2. Approach

Language: Python

Libraries used: Pygame (GUI), collections/custom classes (state), heapq (priority queue for A*)

State Representation: A list of tubes, where each tube is ordered list of colors (bottom to top) Ex: [['R','R','B'], ['B', 'G'], [], ...]

Initial State: File-loaded configuration of tubes with mixed colors

Operators: A valid move is a pour from tube src to tube dst, allowed only when: the top colour of src matches the top colour of dst (or dst is empty), and dst is not full. Each move has a cost of 1.

Heuristics: **Heuristics 1** - Number of tubes not yet complete

Heuristics 2 - Color misplacement: counts total units in incomplete tubes.

Penalises states where colours are spread across many units in wrong tubes.

Heuristics 3 - Within-tube discontinuities: adjacent cells of different colour each require at least one pour to separate and cross-tube fragmentation: for each colour split across k tubes, at least k-1 pours are needed to consolidate it.

3. Implemented Algorithms

Uninformed search:

BFS: Explores level by level using a queue; guaranteed optimal (fewest moves); high memory use since all frontier states are kept

DFS: Explores depth-first using a stack with a depth cap (200); fast but not optimal; can find long or suboptimal solutions

IDDFS: Iteratively deepens DFS from depth 1 upward; optimal like BFS but uses $O(d)$ memory; slow due to repeated state expansions

UCS: Like BFS but orders by cumulative cost $g(n)$; optimal with weighted moves; since all move costs are 1 here it behaves like BFS

3. Implemented Algorithms

Heuristic search:

Greedy Best-First: Orders by $h(n)$ only; fast but not optimal; can be misled by the heuristic

A*: Orders by $f(n) = g(n) + h(n)$; optimal and complete when h is admissible; slower than Greedy but reliable

Weighted A* : Orders by $f(n) = g(n) + W \cdot h(n)$; trades optimality for speed; higher W = faster but potentially longer solutions

4. Experimental Results

	Solved (%)	Avg Moves	Avg States	Avg Memory (kb)	Avg Time (s)
A*	100.00	10	22	32.0	0.010
BFS	90.00	10	2709	2382.0	0.390
DFS	100.00	62	63	190.8	0.010
Greedy	100.00	10	11	16.6	0.000
IDDFS	60.00	6	70471	87.4	1.800
UCS	90.00	10	6191	3392.1	0.570
WA*	100.00	10	11	17.1	0.000

Table 1: Experimental results for low complexity puzzles

4. Experimental Results

	Solved (%)	Avg Moves	Avg States	Avg Memory (kb)	Avg Time (s)
A*	100.00	18	19	71.9	0.010
BFS	60.00	15	26513	30597.0	3.690
DFS	100.00	127	1307	1704.9	0.150
Greedy	100.00	18	19	65.7	0.010
UCS	60.00	15	53703	43930.2	5.070
WA* (W=1.5)	100.00	18	19	77.5	0.020

Table 2: Experimental results for medium complexity puzzles

4. Experimental Results

	Solved (%)	Avg Moves	Avg States	Avg Memory (kb)	Avg Time (s)
A*	100.00	41	69	675.5	0.040
DFS	100.00	143	1558	3032.3	0.130
Greedy	100.00	42	43	449.4	0.030
WA* (W=1.5)	100.00	42	43	441.8	0.030

Table 3: Experimental results for high complexity puzzles

4. Experimental Results

- All algorithms can solve **easy puzzles**, but performance differences are already visible.
- **Heuristic methods (A*, Greedy, WA*) consistently explore far fewer states** than uninformed search.
- **A*** achieves the best balance between **optimal solutions and efficiency**.
- **Greedy and WA*** are faster than **A***, but may sacrifice optimality slightly.
- **DFS produces poor-quality solutions** (very high move counts).
- **BFS, UCS, and IDDFS scale poorly**, with exponential growth in states and memory.
- For **medium and hard puzzles**, **uninformed methods start to become impractical**.
- **Heuristic search is essential for solving complex Water Sort puzzles efficiently**.

5. References

- *Sorting Balls and Water: Equivalence and Computational Complexity*, 19 february 2022 - <https://arxiv.org/pdf/2202.09495>
- *Parallel Branch-and-Bound – Comparing Serial and OpenMP Implementations in a Color Sorting Game*, 2021 - <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9655967&tag=1>
- *Optimization of backtracking algorithm with heuristic strategy for logic-based sorting puzzle game solving*, 4 January 2025 - <https://jutif.if.unsoed.ac.id/index.php/jurnal/article/view/4031>
- Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 2010 - <https://people.engr.tamu.edu/guni/csce625/slides/AI.pdf>